ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК 02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №21»

Выполнила студентка:	Трофимовская Милена Олеговна 54-15 группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководители:	Борисов Егор Николаевич Дадаев М.В.
Оценка	

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ
Заведующий отделением информационных технологий А.Ю. Бычков
_____
«_____» _____ 2026 г.

ЗАДАНИЕ

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	54-15
Ф.И.О. студента	Трофимовская Милена Олеговна

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №21»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта:

Сценарий	Санузел
Датчик 1 (аналоговый)	MQ2 (дым, 0–100 %)
Датчик 2	PIR (присутствие)
Датчик 3	DHT22 (влажность)
Исполнительное устройство	реле (вентилятор)
Светодиод-индикатор (цвет, ток)	красный, 20 мА
Период опроса T	1 с
Порог события	дым > 25 %

Дата выдачи задания для выполнения курсового проекта	«13» октября 2026 г.
Срок защиты курсового проекта	«5» ноября 2026 г.

Содержание

Введение....4
1. Постановка задачи....6
2. Структура и алгоритм работы системы....8
2.1. Структурная схема....8
2.2. Алгоритм функционирования....8
2.3. Протокол обмена данными....9
3. Разработка аппаратной части....11
3.1. Выбор компонентов и подключение....11
3.2. Расчёт резисторов и оценка потребления....12
3.3. Принципиальная схема в EasyEDA....13
3.4. Модель в Wokwi....14
4. Разработка прошивки....15
5. Проектирование базы данных....17
5.1. Модель данных....17
5.2. Описание таблиц....17
5.3. Контрольные запросы....18
6. Загрузка и контроль данных в Node-RED....19
6.1. Состав потока....19
6.2. Функция разбора журнала....20
6.3. Запуск....21
7. Тестирование и результаты....22
7.1. Штатный сценарий....22
7.2. Сценарий неисправности датчика....26
7.3. Повторная загрузка....27
7.4. Сводка результатов....27
8. Расчёт объёма данных....28
9. Управление версиями Git....29
10. Предложения по модернизации....31
Выводы....32
Список используемых сокращений....33
Список используемой литературы....34
Приложение А Структурная схема....36
Приложение Б Схема электрическая принципиальная....37
Приложение В Перечень элементов....38
Приложение Г Модель в Wokwi....39
Приложение Д Листинг программы Arduino (variant_21.ino)....40
Приложение Е Листинг функции разбора журнала Node-RED....42
Приложение Ж Скрипты базы данных....43
Приложение И Журналы Serial, загрузочный скрипт и результаты запросов….44
Приложение К История Git....47

Введение

Интеллектуальные интегрированные системы (ИИС) строятся на основе датчиков, исполнительных устройств и программируемых контроллеров, объединённых каналами передачи данных и средствами хранения информации. Типовая задача сопровождения таких систем состоит в том, чтобы получать показания датчиков, сохранять их, находить отклонения от нормы и выдавать результат в виде, пригодном для анализа.

Прежде чем собирать устройство из реальных компонентов, его проектируют и проверяют на модели. Принципиальную схему разрабатывают в системе автоматизированного проектирования EasyEDA, работу программы микроконтроллера проверяют в симуляторе Wokwi, результаты измерений сохраняют в базе данных SQLite, а за загрузку и контроль данных отвечает среда потоковой разработки Node-RED. Все файлы проекта ведутся в системе контроля версий Git. Такой набор программ не требует установки оборудования и позволяет воспроизвести проект на любом компьютере.

Объектом курсового проекта является виртуальная система сбора и учёта данных на базе платы Arduino Uno для сценария «Санузел». Предмет проекта – схемотехника, программа микроконтроллера, структура базы данных и средства загрузки данных, обеспечивающие контроль задымления, присутствия людей и влажности воздуха в санузле.

Цель проекта – разработать и документировать работающую виртуальную систему, в которой показания трёх датчиков через последовательный порт попадают в журнал, из журнала автоматически загружаются в базу данных и проверяются на выход за допустимые пределы.

Для достижения цели необходимо решить следующие задачи:

- разработать структурную схему и алгоритм работы системы;
- выбрать компоненты, рассчитать токоограничивающие резисторы и выполнить принципиальную схему в EasyEDA по требованиям ЕСКД;
- собрать модель в Wokwi и написать программу для Arduino Uno;
- спроектировать базу данных SQLite и набор контрольных запросов;

– настроить поток Node-RED, который разбирает журнал, загружает данные в базу и фиксирует неисправности;
– провести тестирование на штатном и аварийном сценариях;
– оформить проект в репозитории Git и подготовить пояснительную записку.

1. Постановка задачи

В соответствии с заданием (вариант №21) разрабатывается система контроля санузла. Первый датчик измеряет уровень задымления воздуха, второй определяет присутствие человека, третий измеряет относительную влажность. Исполнительным устройством служит реле, включающее вентилятор. Входные данные варианта приведены в таблице 1.

Таблица 1 — Входные данные варианта №21

Параметр	**Значение**
Сценарий	Санузел
Датчик 1 (аналоговый), s1	MQ2 – дым, диапазон 0–100 %
Датчик 2, s2	PIR (HC-SR501) – присутствие, значения 0 / 1
Датчик 3, s3	DHT22 – относительная влажность воздуха, диапазон 0–100 %
Исполнительное устройство	Реле U4, управляющее вентилятором
Светодиод-индикатор	Красный, рабочий ток 20 мА
Период опроса T	1 с
Порог события	Дым s1 > 25 %

Система должна выполнять следующие функции:

1) каждые T = 1 с опрашивать три датчика и передавать результат в последовательный порт строкой формата «DATA» (значение влажности DHT22 фактически обновляется раз в 2 с, в остальных циклах повторяется прежнее);
2) при превышении уровня дыма 25 % включать реле вентилятора и красный светодиод и передавать строку «EVENT» с типом THRESHOLD; при возврате дыма в норму выключать их и передавать EVENT с типом NORMAL;
3) сохранять строки последовательного порта в текстовый журнал data/log.txt;

4) автоматически загружать журнал в базу SQLite (таблицы readings и events);
5) при значении датчика вне допустимого диапазона или при его отсутствии записывать в таблицу событий неисправность FAULT;
6) выдавать контрольные запросы: десять последних показаний; минимум, среднее и максимум по каждому датчику; список всех событий; число записей в таблицах.

Результатами работы являются: принципиальная схема в EasyEDA, модель в Wokwi, программа Arduino, база данных, поток Node-RED, репозиторий Git и настоящая пояснительная записка с приложениями.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, микроконтроллера Arduino Uno, исполнительного устройства и программной цепочки обработки данных (рисунок 1). Датчики подключены к входам платы; плата управляет реле вентилятора и светодиодом-индикатором и передаёт по последовательному порту (UART) текстовые строки. Строки сохраняются в журнале data/log.txt, откуда Node-RED загружает их в базу данных SQLite.

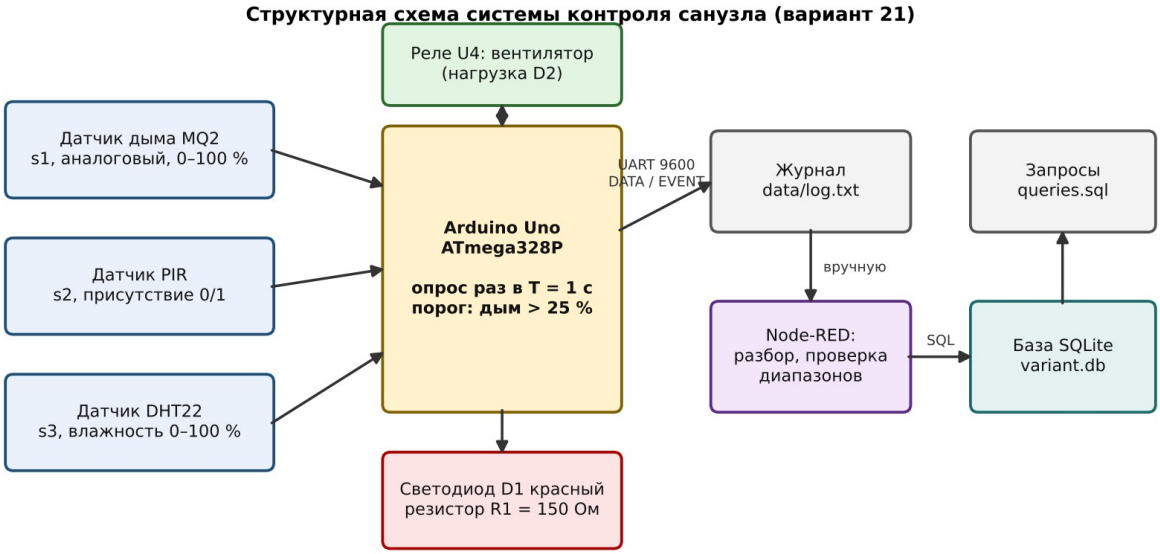


Рисунок 1 — Структурная схема системы

Структурная схема в увеличенном виде приведена в приложении А.

2.2. Алгоритм функционирования

Программа микроконтроллера выполняется циклически (рисунок 2). После инициализации порта и выводов реле и светодиод выключены, номер измерения n = 0, признак превышения порога alarm = ложь. В каждом цикле увеличивается номер измерения, считываются три датчика, и в порт передаётся строка DATA.

Затем проверяется условие события s1 > 25 %. Событие формируется только в момент смены состояния: если условие выполнено, а признак alarm сброшен, включаются реле и светодиод и передаётся EVENT с типом

THRESHOLD; если условие не выполнено, а alarm установлен, исполнительные устройства выключаются и передаётся EVENT с типом NORMAL. Такой подход исключает повторную отправку одного и того же события в каждом цикле. В конце цикла выдерживается пауза T = 1 с.

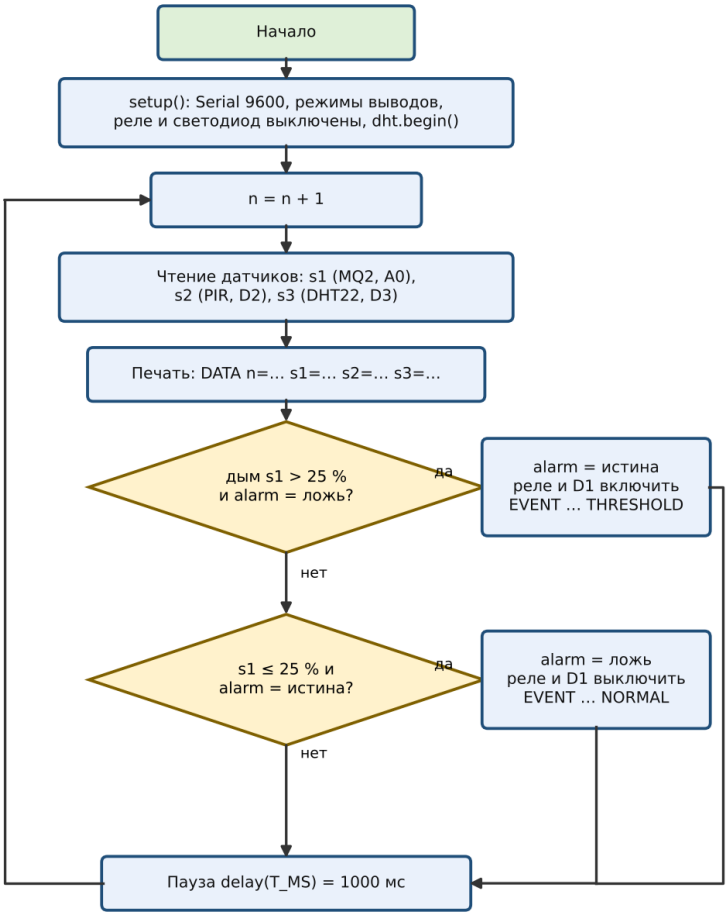


Рисунок 2 — Алгоритм работы программы Arduino

2.3. Протокол обмена данными

Обмен построен на текстовых строках, которые одинаково читаются человеком и разбираются программой. Скорость порта – 9600 бод. Формат строк приведён в таблице 2.

Таблица 2 — Формат строк последовательного порта

Тип	**Формат**	**Пример**
DATA	DATA n=<номер> s1=<дым> s2=<присутствие> s3=<влажность>	DATA n=57 s1=38.82 s2=0 s3=55.0
EVENT	EVENT n=<номер> type=THRESHOLD | NORMAL	EVENT n=57 type=THRESHOLD

Время измерения в базе данных вычисляется по номеру: $t_s = n \cdot T$, где $T = 1$ с. Тип события FAULT программа Arduino не передаёт: его создаёт Node-RED, когда значение датчика отсутствует или выходит за допустимый диапазон (s1: 0–100 %, s2: 0–1, s3: 0–100 %).

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

Для варианта выбраны компоненты, которые есть в библиотеках EasyEDA и Wokwi: плата Arduino Uno R3 на микроконтроллере ATmega328P; модуль датчика дыма и газа MQ2 с аналоговым выходом; пассивный инфракрасный датчик движения HC-SR501 (PIR); датчик влажности и температуры DHT22; одноканальный модуль реле 5 В; красный светодиод индикатора и зелёный светодиод, изображающий нагрузку вентилятора. Подключение к выводам платы приведено в таблице 3.

Таблица 3 — Подключение компонентов к Arduino Uno

Сигнал	**Компонент**	**Вывод**	**Режим**
Дым (s1)	U2, MQ2, выход AO	A0	аналоговый вход
Присутствие (s2)	U3, HC-SR501, выход OUT	D2	цифровой вход
Влажность (s3)	DHT1, DHT22, вывод DATA	D3	цифровой вход/выход
Вентилятор	U4, модуль реле (вход IN)	D7	выход
Индикатор	D1, красный светодиод через R1	D8	выход
Питание	U2, U3, DHT1, U4	5 V, GND	–

Датчик MQ2 выдаёт на выходе AO напряжение, зависящее от концентрации дыма и газов. Десятиразрядный АЦП преобразует напряжение 0–5 В в код 0–1023. Модуль MQ2 даже в чистом воздухе выдаёт ненулевое напряжение (в модели Wokwi код АЦП не опускается ниже 428), поэтому шкала 0–100 % строится не от нуля, а от калибровочного кода чистого воздуха S1_BASE = 428: s1 = 100·(код − 428)/(1023 − 428). Шаг измерения составляет 100/595 ≈ 0,168 %, шаг АЦП по напряжению ΔU = 5 В / 1024 ≈ 4,88 мВ. Порогу 25 % соответствует код 576,75: при коде 576 уровень равен 24,87 % (порог не превышен), при коде 577 – 25,04 % (порог превышен). Без такой калибровки

датчик в симуляторе не опускался бы ниже 41,8 %, и событие NORMAL было бы невозможно.

Датчик PIR при обнаружении движения выдаёт на выходе логическую единицу, при отсутствии – ноль, поэтому считывается функцией digitalRead(). Датчик DHT22 передаёт данные по одному проводу и читается библиотекой DHT sensor library; влажность измеряется с шагом 0,1 %. Минимальный интервал между измерениями DHT22 составляет 2 с, поэтому при общем цикле опроса T = 1 с влажность фактически обновляется один раз в 2 с, то есть в каждом втором цикле; в промежуточных циклах библиотека возвращает прежнее значение, и в журнал повторно записывается то же число. Датчики MQ2 и PIR обновляются в каждом цикле. Влажность изменяется медленно, а порог срабатывания задан по дыму (s1), поэтому такая особенность не влияет на работу системы. Это учтено при анализе журнала (раздел 7) и в предложениях по модернизации.

Нагрузка вентилятора (зелёный светодиод D2 с резистором R2) подключена к контактам реле: общий контакт COM соединён с питанием 5 В, нормально разомкнутый контакт NO – с анодом светодиода. При срабатывании реле нагрузка получает питание.

3.2. Расчёт резисторов и оценка потребления

Резистор R1 ограничивает ток красного светодиода D1, подключённого к выводу D8 (выходное напряжение 5 В). Прямое падение напряжения красного светодиода принимаем равным Uпр = 2,0 В (значение из методических указаний), рабочий ток по заданию I = 20 мА:

$$R1 = (U_{пит} – U_{пр}) / I = (5 – 2{,}0) / 0{,}020 = 150 \text{ Ом}.$$

Расчётное значение 150 Ом совпадает с номиналом стандартного ряда, поэтому округление не требуется. Фактический ток I = (5 – 2,0) / 150 = 20 мА. Мощность, рассеиваемая резистором, $P = I^2 \cdot R = 0{,}02^2 \cdot 150 = 0{,}06$ Вт, достаточно резистора 0,25 Вт. Ток 20 мА не превышает допустимых 40 мА для вывода ATmega328P.

Для зелёного светодиода D2 (Uпр = 2,2 В) принят резистор R2 = 220 Ом:

$$I = (5 – 2{,}2) / 220 \approx 12{,}7 \text{ мА}; P = 0{,}0127^2 \cdot 220 \approx 0{,}035 \text{ Вт}.$$

Оценка потребления от шины 5 В приведена в таблице 4. Суммарный ток около 306 мА, что меньше 500 мА, допустимых для порта USB, поэтому внешний источник питания не требуется. Основной вклад вносит нагреватель датчика MQ2.

Таблица 4 — Оценка потребляемого тока

Потребитель	**Ток, мА**	**Примечание**
Плата Arduino Uno (ATmega328P)	50	типовое
Датчик MQ2, нагреватель	150	по документации
Датчик движения HC-SR501	0,05	< 65 мкА
Датчик DHT22	1,5	при измерении
Модуль реле, катушка	72	при включении
Светодиод D1 с резистором R1	20	расчёт
Светодиод D2 с резистором R2	12,7	расчёт
Итого	≈ 306	< 500 (USB)

3.3. Принципиальная схема в EasyEDA

Схема электрическая принципиальная выполнена в EasyEDA на листе формата А4 в рамке с полями 20 мм слева и 5 мм с остальных сторон, основная надпись – форма 1 по ГОСТ Р 2.104–2023 (шифр документа КП. 5415. 21 ЭЗ). Условные графические обозначения соответствуют ГОСТ 2.721-74 и ГОСТ 2.730-73, позиционные обозначения – ГОСТ 2.710-81: U1 – плата Arduino Uno, U2 – датчик дыма MQ2, U3 – датчик движения, DHT1 – датчик влажности DHT22, D1, D2 – светодиоды, U4 – реле, R1, R2 – резисторы. Позиционные обозначения на схеме и в перечне едины. Схема приведена в приложении Б, перечень элементов (шифр КП. 5415. 21 ПЭ3) – в приложении В.

Порядок построения схемы: сначала в схему вводятся компоненты из библиотеки и задаются позиционные обозначения; затем проводятся цепи питания (5 V, GND) и сигнальные цепи в соответствии с таблицей 3; сигнальные цепи s1–s5 показаны сетевыми метками (s1 – выход PIR к D2, s2 – D8 к

светодиоду D1, s3 – D7 к входу реле, s4 – контакт NO реле к светодиоду D2, s5 – сигнал DHT22 к D3), неиспользуемые выводы (D0 датчика MQ2) отмечены знаком «нет соединения»; после этого выполняется проверка правил (Design Manager → ERC), и схема экспортируется в PDF и PNG.

3.4. Модель в Wokwi

После утверждения схемы выполнена модель в симуляторе Wokwi (приложение Г). В проект добавлены элементы wokwi-arduino-uno, wokwi-gas-sensor (MQ2), wokwi-pir-motion-sensor, wokwi-dht22, wokwi-relay-module, wokwi-led (два) и два резистора. Соединения повторяют схему EasyEDA, файл проекта сохранён в sim/diagram.json. Для работы с DHT22 в проект подключена библиотека DHT sensor library. Уровень дыма задаётся ползунком датчика MQ2, присутствие – ручным включением датчика PIR, влажность – полем «Humidity» датчика DHT22.

Wokwi работает в браузере и не может записывать файлы на диск компьютера. Поэтому строки, выводимые в Serial Monitor, копируются и сохраняются в файл data/log.txt вручную; дальнейшая загрузка в базу данных выполняется автоматически (раздел 6).

4. Разработка прошивки

Программа для Arduino Uno написана на языке C++ в среде Wokwi; полный текст приведён в приложении Д. Параметры варианта вынесены в константы в начале программы: номера выводов, период опроса T_MS = 1000 мс, диапазон датчика дыма S1_MIN = 0 и S1_MAX = 100, порог THRESHOLD = 25,0 %. Для другого варианта достаточно изменить эти константы и три функции чтения датчиков.

Структура программы определяется набором функций (таблица 5).

Таблица 5 — Функции программы

Функция	**Назначение**
setup()	настройка порта (9600 бод) и режимов выводов, запуск DHT22, начальное выключение реле и светодиода
loop()	основной цикл: опрос датчиков, вывод DATA, проверка порога, вывод EVENT, пауза T
readS1()	чтение дыма: analogRead(A0), пересчёт кода АЦП (от S1_BASE = 428 до 1023) в диапазон 0–100 %
readS2()	чтение датчика присутствия digitalRead(D2)
readS3()	чтение влажности dht.readHumidity(); при сбое датчика возвращает NaN
actuatorOn() / actuatorOff()	включение и выключение реле вентилятора
conditionMet()	условие события: s1 > THRESHOLD

Для передачи данных использованы функции Serial.print() и Serial.println(): дым выводится с двумя знаками после запятой, присутствие – целым числом, влажность – с одним знаком. Такой формат совпадает с протоколом из таблицы 2 и разбирается в Node-RED без дополнительных преобразований. Если датчик DHT22 не отвечает, функция readHumidity() возвращает NaN, и в журнал попадает значение «nan»; такая строка будет распознана Node-RED как неисправность датчика.

Пауза между измерениями реализована функцией delay(T_MS). Для данной задачи это допустимо: нет других действий, которые нужно выполнять в

промежутке между опросами, а точность периода в пределах долей секунды достаточна для расчёта времени $t_s = n \cdot T$.

5. Проектирование базы данных

5.1. Модель данных

Для хранения данных выбрана встраиваемая СУБД SQLite: база целиком хранится в одном файле db/variant.db, не требует сервера и работает под управлением консольной программы sqlite3. Модель состоит из двух таблиц (рисунок 3): readings – показания датчиков, events – события и неисправности.

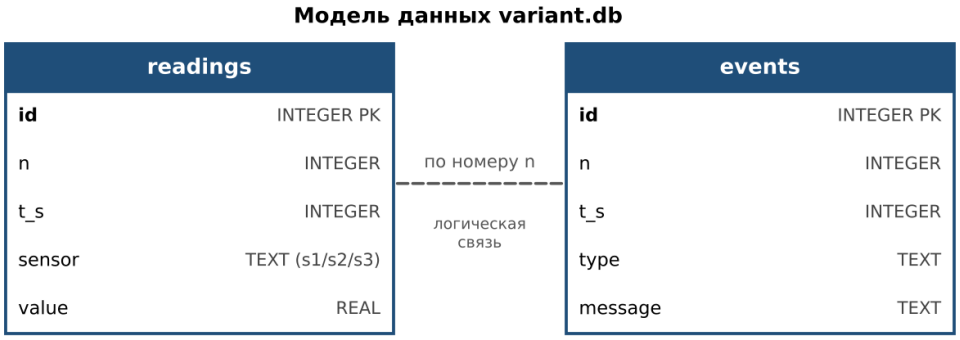


Рисунок 3 — Модель данных

Показания хранятся в «длинном» формате: один датчик в одном цикле – одна строка. Благодаря этому структура таблицы не зависит от числа датчиков, а запросы по конкретному датчику выполняются условием по полю sensor. Связь между таблицами логическая: по номеру измерения n.

5.2. Описание таблиц

Таблица 6 — Структура таблицы readings

Поле	**Тип**	**Назначение**
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения из журнала
t_s	INTEGER, NOT NULL	время от начала измерений, с (t_s = n·T)
sensor	TEXT, NOT NULL	датчик: s1, s2 или s3
value	REAL, NOT NULL	значение датчика

Таблица 7 — Структура таблицы events

Поле	**Тип**	**Назначение**
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения, при котором возникло событие
t_s	INTEGER, NOT NULL	время, с
type	TEXT, NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	пояснение (для FAULT – какой датчик неисправен)

Скрипт создания таблиц db/schema.sql использует оператор CREATE TABLE IF NOT EXISTS, поэтому его можно выполнять повторно без ошибок. Текст скрипта приведён в приложении Ж.

5.3. Контрольные запросы

Для проверки данных составлены четыре запроса (файл db/queries.sql, приложение Ж). Запрос 1 выводит десять последних показаний и показывает, что загрузка дошла до конца журнала. Запрос 2 вычисляет минимум, среднее и максимум по каждому датчику с помощью агрегатных функций MIN, AVG, MAX и группировки GROUP BY sensor; результат позволяет быстро заметить значения, не похожие на реальные. Запрос 3 выводит все события по порядку их возникновения. Запрос 4 подсчитывает число записей в каждой таблице.

6. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Загрузка данных выполнена потоком Node-RED, показанным на рисунке 4. Поток следит за файлом data/log.txt и при каждом сохранении файла автоматически перезагружает базу данных. Дополнительно предусмотрена кнопка ручного запуска. Назначение узлов приведено в таблице 8. Полный экспорт потока (файл nodered/flow.json) размещён в репозитории проекта.

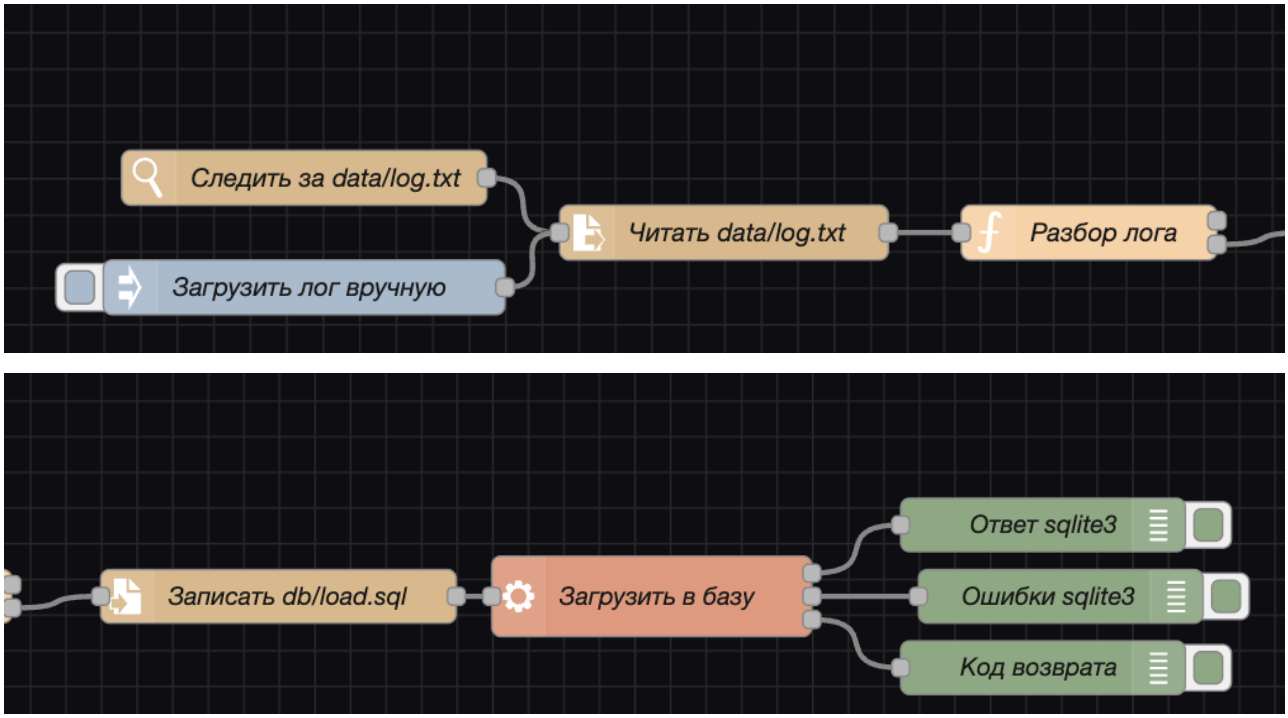


Рисунок 4 — Поток Node-RED в редакторе (вверху – чтение и разбор журнала, внизу – запись и загрузка в базу)

Таблица 8 — Узлы потока

Узел	**Тип**	**Настройка и назначение**
Следить за data/log.txt	watch	формирует сообщение при изменении файла журнала
Загрузить лог вручную	inject	ручной запуск потока кнопкой
Читать data/log.txt	file in	читает файл целиком одной строкой (utf8)
Разбор лога	function	разбирает строки, проверяет диапазоны, формирует SQL; два выхода
Записать db/load.sql	file	пишет SQL-скрипт загрузки, режим «перезаписать файл»
Загрузить в базу	exec	sqlite3 db/variant.db ".read db/schema.sql" ".read db/load.sql"; флажок Append msg.payload снят
Ответ sqlite3	debug	вывод штатного ответа программы
Ошибки sqlite3	debug	вывод ошибок (второй и третий выходы exec)

6.2. Функция разбора журнала

Функция (приложение Е) получает текст журнала и делит его на строки. Каждая строка разбивается по пробелам на тип сообщения (DATA или EVENT) и пары «ключ=значение». Номер измерения n преобразуется в число, время вычисляется как t_s = n·1. Для строки DATA для каждого из датчиков s1, s2, s3 формируется команда INSERT в таблицу readings. Значение проверяется по допустимому диапазону; если оно отсутствует (nan) или выходит за границы, добавляется команда INSERT в таблицу events с типом FAULT и текстом «датчик sN вне диапазона». Для отсутствующего значения запись в readings не создаётся, так как поле value не допускает пустых значений. Для строки EVENT формируется запись в events с типом, переданным контроллером.

Функция имеет два выхода. Первый оставлен свободным, второй выдаёт единый скрипт load.sql, который начинается с команд BEGIN и очистки таблиц (DELETE FROM readings, DELETE FROM events, сброс счётчиков sqlite_sequence) и заканчивается командой COMMIT. Очистка делает загрузку повторяемой: при повторной загрузке того же журнала записи не дублируются, а

транзакция гарантирует, что база не останется в наполовину загруженном состоянии.

6.3. Запуск

Работа ведётся на компьютере без прав администратора, все программы портативные и лежат в папке %USERPROFILE%\tools. Командный файл start-nodered.bat перед запуском переходит в папку проекта, благодаря чему относительные пути data/log.txt и db/variant.db указываются от корня проекта. Порядок запуска: выполнить start-nodered.bat, открыть редактор по адресу http://localhost:1880, импортировать поток из файла nodered/flow.json и нажать «Deploy». После сохранения файла data/log.txt загрузка происходит автоматически; результат проверяется командой:

```
sqlite3 db/variant.db ".read db/queries.sql"
```

7. Тестирование и результаты

7.1. Штатный сценарий

Сценарий выполнен в симуляторе Wokwi: уровень дыма, присутствие и влажность изменялись вручную элементами управления датчиков, а строки Serial Monitor сохранены без изменений в файл data/log.txt (91 цикл, 91 с). Моделируемая ситуация такова. В цикле 1 ползунок MQ2 стоит в положении по умолчанию (400 ppm), дым выше порога (80,50 %): формируется событие THRESHOLD, реле вентилятора и светодиод включены. В цикле 11 концентрация снижена до 0,1 ppm, s1 = 0,00 %: формируется событие NORMAL, вентилятор и светодиод выключаются (рисунок 6). В циклах 31–35 срабатывает датчик PIR (s2 = 1) – в помещении человек. В цикле 57 дым повышен до 11 ppm, s1 = 38,82 %: снова THRESHOLD, реле и светодиод включаются (рисунок 5). С цикла 67 влажность увеличена до 93,5 % (датчик DHT22 обновляет значение не чаще раза в 2 с, поэтому изменение влажности может проявиться в журнале с задержкой до одного цикла). В цикле 78 дым снижен, s1 = 0,00 %: событие NORMAL (рисунок 7). Журнал приведён в приложении И, в таблице 9 показаны характерные циклы.

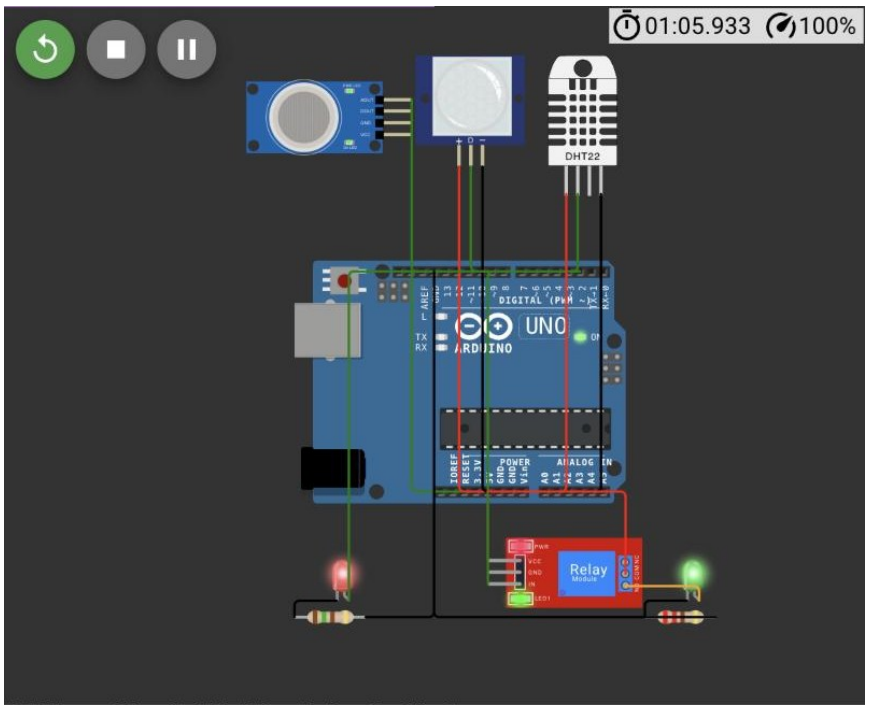


Рисунок 5 — Модель Wokwi: дым выше порога, реле и светодиод включены

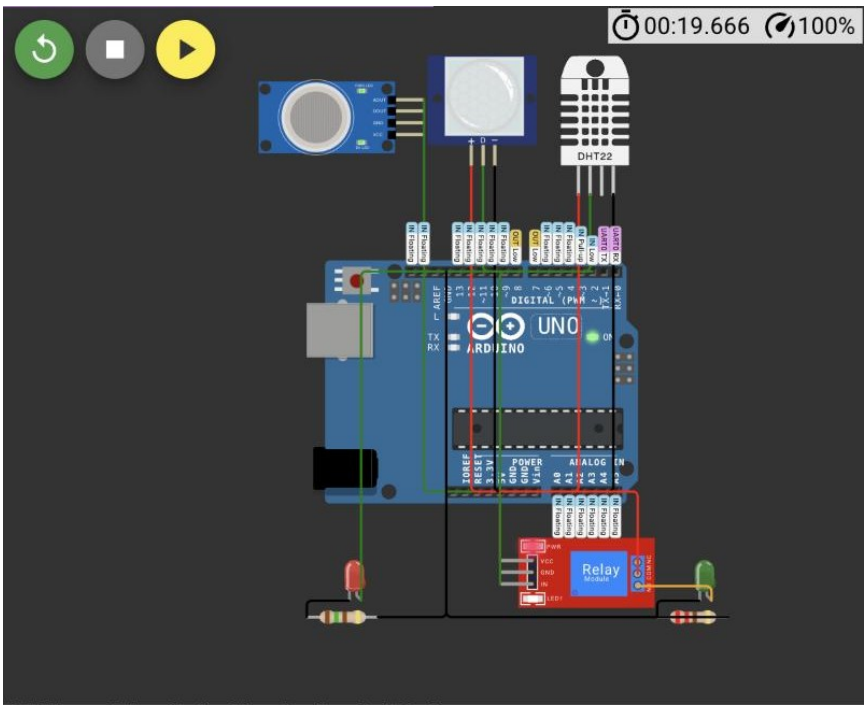


Рисунок 6 — Модель Wokwi: дым в норме, реле и светодиод выключены

```
DATA n=74 s1=38.82 s2=0 s3=93.5
DATA n=75 s1=38.82 s2=0 s3=93.5
DATA n=76 s1=38.82 s2=0 s3=93.5
DATA n=77 s1=38.82 s2=0 s3=93.5
DATA n=78 s1=0.00 s2=0 s3=93.5
EVENT n=78 type=NORMAL
```

Рисунок 7 — Serial Monitor: события THRESHOLD и NORMAL

Таблица 9 — Характерные циклы журнала (полный журнал – в приложении И)

n	**t_s, с**	**s1, %**	**s2**	**s3, %**	**Событие**
1	1	80,50	0	55,0	THRESHOLD
2	2	80,50	0	55,0	–
10	10	80,50	0	55,0	–
11	11	0,00	0	55,0	NORMAL
12	12	0,00	0	55,0	–
30	30	0,00	0	55,0	–
31	31	0,00	1	55,0	–
35	35	0,00	1	55,0	–
36	36	0,00	0	55,0	–
56	56	0,00	0	55,0	–

Продолжение таблицы 9

n	t_s, c	s1, %	s2	s3, %	Событие
57	57	38,82	0	55,0	THRESHOLD
58	58	38,82	0	55,0	–
66	66	38,82	0	55,0	–
67	67	38,82	0	93,5	–
77	77	38,82	0	93,5	–
78	78	0,00	0	93,5	NORMAL
79	79	0,00	0	93,5	–
91	91	0,00	0	93,5	–

После сохранения журнала Node-RED загрузил данные в базу. Счётчики записей: в таблице readings 273 строки (91 цикл по 3 датчика), в таблице events – 4 строки. Результат загрузки в панели debug показан на рисунке 8, результаты контрольных запросов приведены в таблицах 10–13 и на рисунке 9.

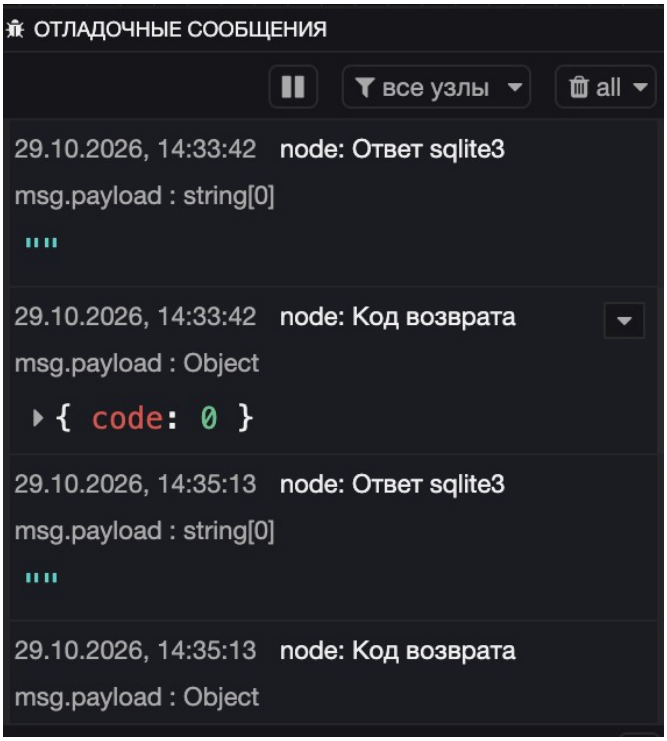


Рисунок 8 — Панель debug Node-RED: загрузка выполнена, код возврата 0

Таблица 10 — Запрос 1: десять последних показаний

id	n	t_s	sensor	value
273	91	91	s3	93,5
272	91	91	s2	0
271	91	91	s1	0
270	90	90	s3	93,5
269	90	90	s2	0
268	90	90	s1	0
267	89	89	s3	93,5
266	89	89	s2	0
265	89	89	s1	0
264	88	88	s3	93,5

Таблица 11 — Запрос 2: минимум, среднее и максимум

sensor	min_v	avg_v	max_v
s1	0	17,8	80,5
s2	0	0,05	1
s3	55	65,58	93,5

Значения лежат в допустимых диапазонах: дым 0–80,50 %, присутствие 0–1, влажность 55,0–93,5 %. Среднее значение s2 равно 0,05: присутствие зафиксировано в 5 циклах из 91.

Таблица 12 — Запрос 3: все события

id	n	t_s	type	message
1	1	1	THRESHOLD	–
2	11	11	NORMAL	–
3	57	57	THRESHOLD	–
4	78	78	NORMAL	–

События совпали с моделируемой ситуацией: THRESHOLD в циклах 1 и 57 (t_s = 1 и 57 с), NORMAL в циклах 11 и 78 (t_s = 11 и 78 с). Неисправностей FAULT нет.

Таблица 13 — Запрос 4: количество записей

Таблица	**Записей**
readings	273
events	4

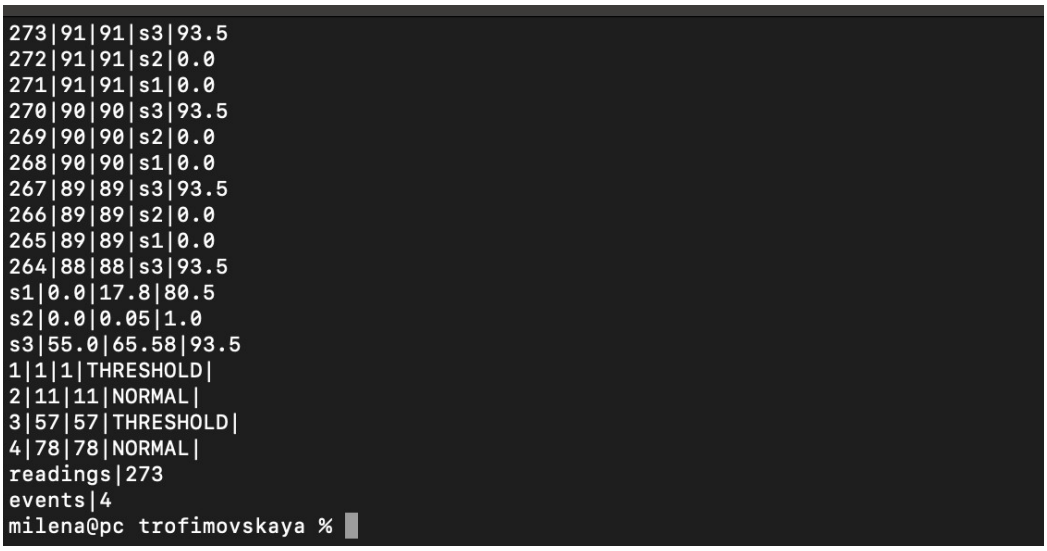


Рисунок 9 — Результат контрольных запросов в терминале

7.2. Сценарий неисправности датчика

Для проверки контроля диапазонов в журнал подана строка, в которой датчик дыма передаёт недопустимое значение 999 (диапазон 0–100 %):

```
DATA n=1 s1=999 s2=1 s3=55.0
```

После загрузки в таблице readings появились 3 строки (по одной на каждый датчик), а в таблице events – 1 строка: n = 1, t_s = 1, type = FAULT, message = «датчик s1 вне диапазона». Данные других датчиков (s2 = 1, s3 = 55,0 %) сохранены в штатном порядке. Файл load.sql для этого теста приведён в приложении И, результат проверки показан на рисунке 10.

Дополнительно проверен отказ датчика: при строке DATA n=1 s1=nan s2=0 s3=50.0 (значение отсутствует) в readings записываются две строки, а в events – событие FAULT для датчика s1.

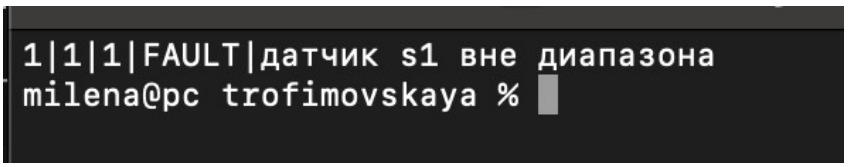


Рисунок 10 — Событие FAULT в таблице events после подстановки тестового журнала

7.3. Повторная загрузка

Журнал штатного сценария загружен три раза подряд без изменений. Счётчики остались прежними: 273 строки в readings и 4 строки в events. Это подтверждает, что очистка таблиц в начале скрипта load.sql предотвращает дублирование данных.

7.4. Сводка результатов

Таблица 14 — Результаты проверок

№	Проверка	Ожидаемый результат	Получено	Итог
1	Загрузка штатного журнала	273 показания, 4 события	273 и 4	пройден
2	Событие при s1 > 25 %	THRESHOLD при s1 > 25 % (n = 1, 57)	n = 1 и 57	пройден
3	Возврат дыма в норму	NORMAL при s1 ≤ 25 % (n = 11, 78)	n = 11 и 78	пройден
4	Значение вне диапазона (s1 = 999)	событие FAULT, записаны 3 показания	В events 1 строка: n = 1, t_s = 1, FAULT, «датчик s1 вне диапазона»; в readings 3 строки	пройден
5	Отсутствие значения (s1 = nan)	событие FAULT, записаны 2 показания	В events 1 строка FAULT «датчик s1 вне диапазона»; в readings 2 строки	пройден
6	Повторная загрузка	счётчики не растут	273 и 4	пройден
7	Включение U4 и D1 в Wokwi	включены при s1 > 25 %	включаются при THRESHOLD, выключаются при NORMAL	пройден

8. Расчёт объёма данных

Оценка объёма нужна для выбора носителя и срока хранения. Расчёт для непрерывной работы системы с периодом T = 1 с приведён в таблице 15.

Таблица 15 — Объём данных

Показатель	**Расчёт**	**Результат**
Циклов в сутки	86 400 с / 1 с	86 400
Записей readings в сутки	86 400 · 3 датчика	259 200
Размер строки в базе (оценка)	id, n, t_s, sensor, value	≈ 40 Б
Объём базы за сутки	259 200 · 40 Б	≈ 10,4 МБ
Размер строки журнала	DATA n=86400 s1=100.00 s2=1 s3=100.0 + CRLF	≈ 38 Б
Объём журнала за сутки	86 400 · 38 Б	≈ 3,3 МБ
Объём базы за 30 суток	10,4 МБ · 30	≈ 311 МБ
Объём базы за год	10,4 МБ · 365	≈ 3,8 ГБ

Объём базы за год составляет около 3,8 ГБ, что для SQLite не является проблемой: ограничения СУБД намного выше, а практический предел определяется свободным местом на диске. При длительной эксплуатации рекомендуется архивировать журнал и периодически удалять показания старше заданного срока. Если увеличить период опроса до T = 2 с, что соответствует минимальному интервалу измерений DHT22, объём данных уменьшится вдвое.

9. Управление версиями Git

Проект ведётся в публичном репозитории Git с названием kyrsovaya21, опубликованном на GitHub по адресу https://github.com/volan11/kyrsovaya21. В репозитории размещены архив проекта, исходный текст прошивки, модель Wokwi, материалы EasyEDA (принципиальная схема и перечень элементов BOM), поток Node-RED, SQL-скрипты, журналы, инструкция пользователя, README и остальные файлы проекта. Структура репозитория приведена в таблице 16. Файл базы db/variant.db и промежуточный db/load.sql создаются автоматически, поэтому в репозиторий не включаются (файл .gitignore).

Таблица 16 — Структура репозитория

Каталог или файл	**Содержимое**
README.md	описание проекта, порядок запуска, ссылки
firmware/variant_21/	исходный текст прошивки Arduino (variant_21.ino)
sim/	модель Wokwi: diagram.json, sketch.ino, libraries.txt
easyeda/	материалы EasyEDA: исходный файл проекта (JSON), принципиальная схема (PNG и PDF с рамкой), перечень элементов (bom.csv)
db/	schema.sql, queries.sql, queries_result.txt (результаты запросов)
nodered/	flow.json (полный экспорт потока), function_parse.js
data/	журналы: log.txt (штатный сценарий), log_fault.txt (неисправность)
docs/	записка (DOCX, PDF), презентация, речь, инструкция пользователя, рисунки
archive/	архив проекта kyrsovaya21_v1.0.zip
start-nodered.bat, env.bat, .gitignore	запуск Node-RED на Windows и правила Git

Модель Wokwi опубликована по ссылке: https://wokwi.com/projects/479205845917443073. Файлы модели (sim/diagram.json, sim/sketch.ino) хранятся в репозитории.

Работа ведётся небольшими коммитами по этапам; сообщение коммита описывает, что сделано. Порядок работы:

```
git init
git config user.name "Milena Trofimovskaya"
git add .
git commit -m "Структура проекта"
... (по одному коммиту на этап)
git tag v1.0
git log --oneline
git remote add origin
https://github.com/volan11/kyrsovaya21.git
git push -u origin main --tags
```

История разработки подтверждена пятью осмысленными коммитами по этапам работы (таблица 17, приложение К). Метка v1.0 поставлена на финальной версии, сданной на защиту.

Таблица 17 — Коммиты проекта

Хеш	**Дата**	**Сообщение**
77674d1	29.10.2026	Структура проекта, README, схема электрическая принципиальная и перечень элементов
34219fa	29.10.2026	Модель Wokwi, прошивка variant_21.ino и журнал работы системы
a2e7ce8	29.10.2026	Схема базы данных SQLite и поток Node-RED для загрузки журнала
1dbb28f	29.10.2026	Журнал тестов и проверка неисправности FAULT
—	29.10.2026	Записка, презентация, речь, архив, метка v1.0

10. Предложения по модернизации

1) увеличить период опроса до 2 с или читать DHT22 отдельно, чтобы каждое значение влажности было новым (сейчас при T = 1 с влажность обновляется раз в 2 с);
2) откалибровать нуль шкалы MQ2 по реальному датчику в чистом воздухе (в модели принят код 428) и добавить гистерезис порога, чтобы уменьшить частые переключения;
3) перейти к опросу по millis(), чтобы точнее выдерживать период T, и сделать загрузку в базу накопительной с защитой от дублей вместо замены снимка журнала;

Выводы

В ходе курсового проекта разработана и проверена виртуальная система сбора и учёта данных для сценария «Санузел» (вариант №21). Получены следующие результаты:

- выполнена структурная схема и алгоритм работы, определён текстовый протокол обмена с контроллером;
- подобраны компоненты, рассчитаны резисторы R1 = 150 Ом и R2 = 220 Ом, суммарное потребление около 306 мА не превышает возможностей порта USB;
- принципиальная схема выполнена в EasyEDA по требованиям ЕСКД, модель собрана в Wokwi;
- написана программа для Arduino Uno, которая опрашивает три датчика каждую секунду и формирует события при дыме выше 25 % только в момент смены состояния;
- спроектирована база данных SQLite из двух таблиц и подготовлены контрольные запросы;
- настроен поток Node-RED, автоматически и повторяемо загружающий журнал в базу и фиксирующий неисправности датчиков;
- тестирование на штатном и аварийном сценариях подтвердило правильность работы: 273 показания и 4 события в штатном режиме, событие FAULT при недопустимом значении;
- оценён объём данных – около 10,4 МБ в сутки, проект оформлен в репозитории Git.

Разработанный проект может быть использован как основа для других сценариев: при смене датчиков достаточно изменить константы и функции чтения в программе, диапазоны и период в функции Node-RED, а структура базы данных остаётся прежней.

Список используемых сокращений

АЦП	аналого-цифровой преобразователь
БД	база данных
ГОСТ	государственный стандарт
ЕСКД	единая система конструкторской документации
ИИС	интеллектуальные интегрированные системы
СУБД	система управления базами данных
DHT22	цифровой датчик влажности и температуры
ERC	проверка электрических правил (Electrical Rules Check)
MQ2	полупроводниковый датчик дыма и горючих газов
PIR	пассивный инфракрасный датчик движения (Passive Infrared)
SQL	язык структурированных запросов (Structured Query Language)
UART	универсальный асинхронный приёмопередатчик
USB	универсальная последовательная шина

Список используемой литературы

1. Петин В.А. Arduino и Raspberry Pi в проектах Internet of Things. – СПб.: БХВ-Петербург, 2016. – 320 с.
2. Монк С. Программируем Arduino. Профессиональная работа со скетчами. – СПб.: Питер, 2017. – 288 с.
3. Чакон С., Штрауб Б. Pro Git. – 2-е изд. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 20.10.2026).
4. ГОСТ Р 2.104–2023. Единая система конструкторской документации. Основные надписи. Введён 01.03.2024.
5. ГОСТ 2.702–2011. ЕСКД. Правила выполнения электрических схем. – М.: Стандартинформ, 2012.
6. ГОСТ 2.301–68. ЕСКД. Форматы. – М.: Издательство стандартов, 1968.
7. ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования к выполнению. – М.: Стандартинформ, 2009.
8. ГОСТ 2.710-81. ЕСКД. Обозначения буквенно-цифровые в электрических схемах. – М.: Издательство стандартов, 1981.
9. ГОСТ 2.721-74. ЕСКД. Обозначения условные графические в схемах. Обозначения общего применения. – М.: Издательство стандартов, 1974.
10. Документация Arduino: Language Reference. – URL: https://www.arduino.cc/reference/en/ (дата обращения: 20.10.2026).
11. Документация Wokwi. – URL: https://docs.wokwi.com/ (дата обращения: 20.10.2026).
12. Проект Wokwi варианта №21. – URL: https://wokwi.com/projects/479205845917443073.
13. Документация Node-RED. – URL: https://nodered.org/docs/ (дата обращения: 20.10.2026).
14. Документация SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 20.10.2026).

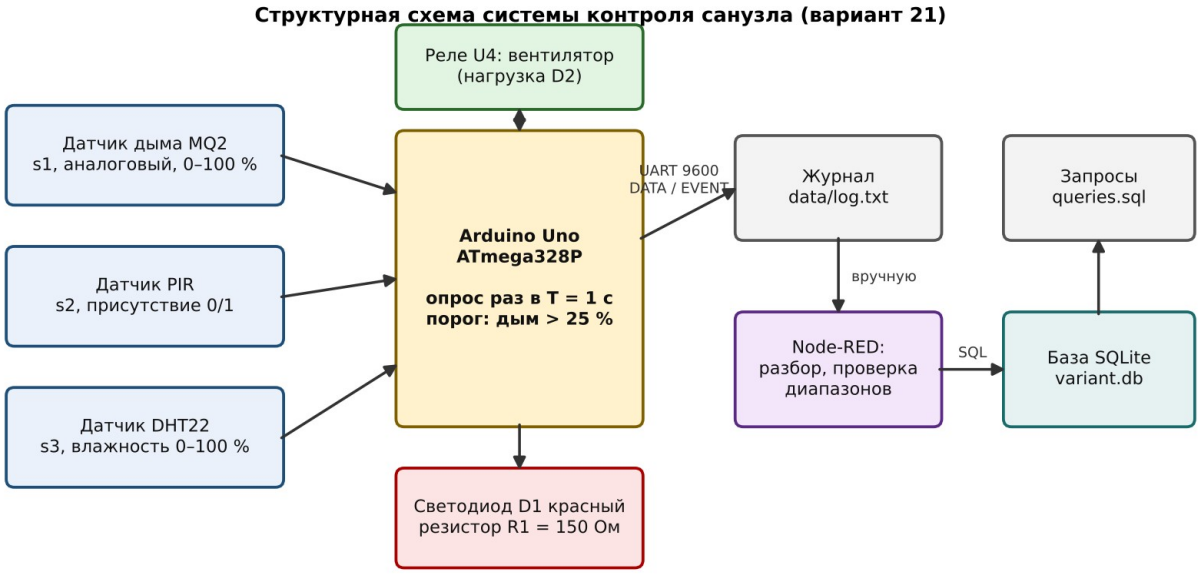
15. Документация EasyEDA. – URL: https://docs.easyeda.com/ (дата обращения: 20.10.2026).

16. Микроконтроллер ATmega328P. Datasheet. – Microchip Technology Inc. – URL: https://www.microchip.com (дата обращения: 20.10.2026).

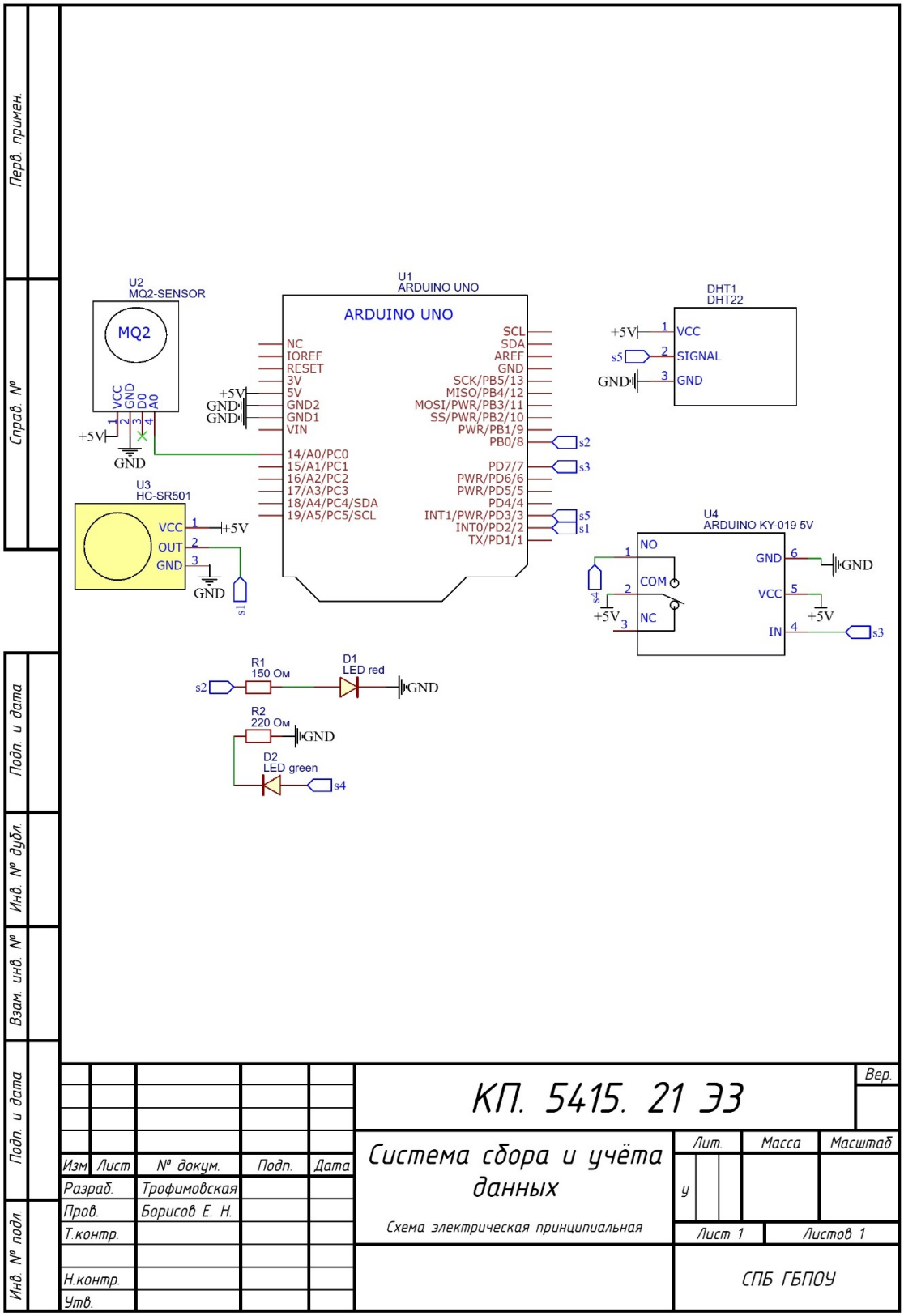
17. Adafruit. DHT sensor library. – URL: https://github.com/adafruit/DHT-sensor-library (дата обращения: 20.10.2026).

18. Трофимовская М. Курсовой проект «Система сбора и учёта данных, вариант 21». – URL: https://github.com/volan11/kyrsovaya21 (дата обращения: 29.10.2026).

Приложение А Структурная схема



Приложение Б Схема электрическая принципиальная



					КП. 5415. 21 ЭЗ		Вер.
Изм	Лист	№ докум.	Подп.	Дата	Система сбора и учёта данных	Лит.	Масса / Масштаб
Разраб.		Трофимовская				у	
Пров.		Борисов Е. Н.			Схема электрическая принципиальная	Лист 1	Листов 1
Т.контр.							
Н.контр.						СПБ ГБПОУ	
Утв.							

Перв. примен. | Справ. № | Подп. и дата | Инв. № дубл. | Взам. инв. № | Подп. и дата | Инв. № подл.

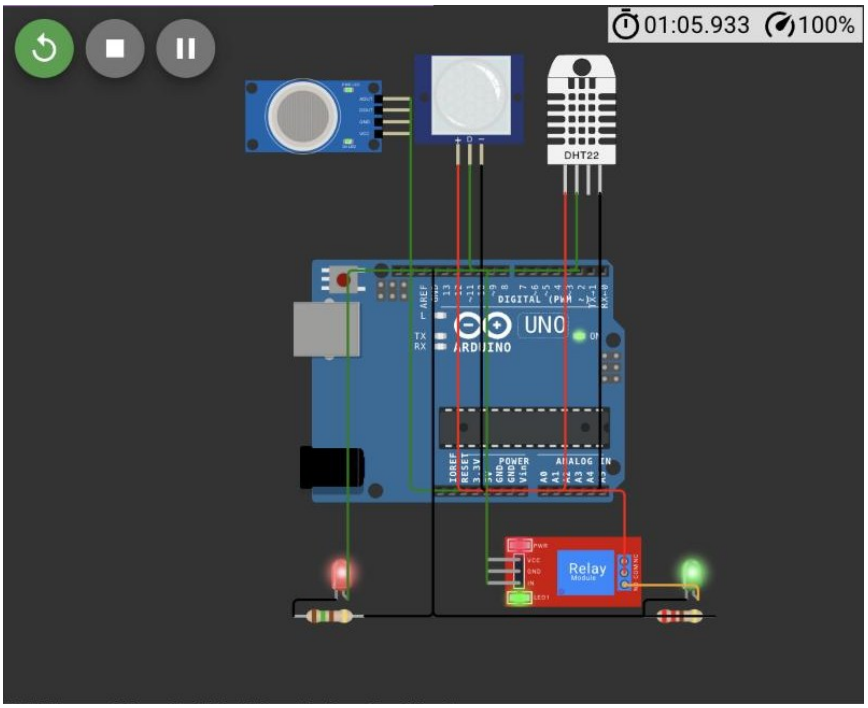
Приложение В Перечень элементов

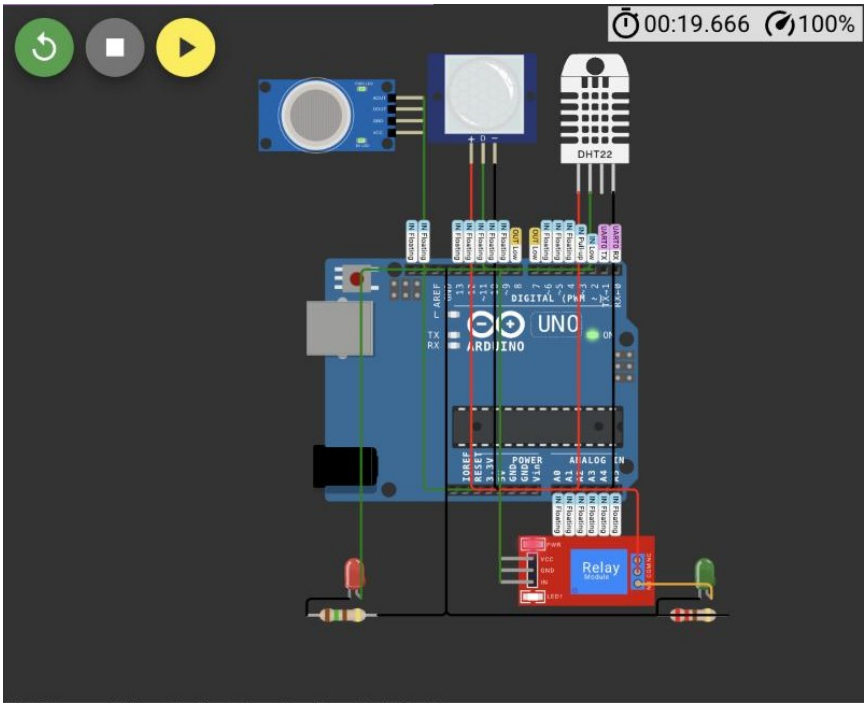
Поз. обозначение	Наименование	Кол.	Примечание
	Диоды, тиристоры, стабилитроны		
D1	LED red	1	LED_0603
D2	LED green	1	LED_0603
	Датчики		
DHT1	DHT22	1	DHT22
	Резисторы, потенциометры		
R1	150 Ом	1	R0603
R2	220 Ом	1	R0603
	Микросхемы, неразборные узлы		
U1	ARDUINO UNO	1	ARDUINO_UNO_SHIEL
U2	MQ2-SENSOR	1	MQ-2-H
U3	HC-SR501	1	HC-SR501
U4	ARDUINO KY-019 5V	1	ARDUINO RELAY KY-0

Перв. примен. | Справ. № | Подп. и дата | Инв. № дубл. | Взам. инв. № | Подп. и дата | Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата	КП. 5415. 21 ПЭЗ	Вер.
Разраб.		Трофимовская			Система сбора и учёта данных	Лит. у	Лист 1	Листов 1
Пров.		Борисов Е. Н.				
Нач. сект.						СПБ ГБПОУ
Н.контр.					Перечень элементов	
Утв.						

Приложение Г Модель в Wokwi





Ссылка на проект Wokwi: https://wokwi.com/projects/479205845917443073

Репозиторий проекта на GitHub: https://github.com/volan11/kyrsovaya21

Приложение Д Листинг программы Arduino (variant_21.ino)

```
// Вариант 21. Санузел. Система контроля задымления и влажности
// Плата: Arduino Uno (ATmega328P), симулятор Wokwi
// Подключение:
//   Датчик 1 (MQ2, дым 0-100 %)            -> A0 (аналоговый выход AOUT)
//   Датчик 2 (PIR, присутствие 0/1)        -> D2
//   Датчик 3 (DHT22, влажность воздуха)    -> D3
//   Реле U4 (вентилятор)                   -> D7
//   Светодиод D1 (красный) через R1=150 Ом -> D8
// Библиотека: DHT sensor library (Adafruit), см. sim/libraries.txt

#include <DHT.h>

const int PIN_S1    = A0;   // датчик 1: MQ2 (дым)
const int PIN_S2    = 2;    // датчик 2: PIR (присутствие)
const int PIN_S3    = 3;    // датчик 3: DHT22 (влажность)
const int PIN_RELAY = 7;    // реле вентилятора
const int PIN_LED   = 8;    // светодиод-индикатор

const unsigned long T_MS = 1000;  // период опроса T = 1 с
const float S1_MIN = 0.0;         // диапазон датчика 1: 0...100 %
const float S1_MAX = 100.0;
const float THRESHOLD = 25.0;     // порог: дым > 25 %
const int   S1_BASE = 428;        // код АЦП MQ2 в чистом воздухе (калибровка нуля)

DHT dht(PIN_S3, DHT22);

unsigned long n = 0;    // номер измерения
bool alarm = false;     // порог сейчас превышен?

// Датчик 1: MQ2 (дым), пересчёт кода АЦП в проценты.
// MQ2 в чистом воздухе выдаёт ненулевое напряжение (код S1_BASE), поэтому
// шкала 0...100 % строится от S1_BASE до 1023: s1 = мин + (макс - мин) * (код -
S1_BASE) / (1023 - S1_BASE)
float readS1() {
  int raw = analogRead(PIN_S1);
  if (raw < S1_BASE) raw = S1_BASE;
  return S1_MIN + (S1_MAX - S1_MIN) * (raw - S1_BASE) / (1023.0 - S1_BASE);
}

// Датчик 2: PIR (присутствие), 1 - движение есть
float readS2() {
  return digitalRead(PIN_S2);
}

// Датчик 3: DHT22 (относительная влажность, %)
float readS3() {
  return dht.readHumidity();   // при сбое вернёт NaN
}

void actuatorOn() {
  digitalWrite(PIN_RELAY, HIGH);
}

void actuatorOff() {
  digitalWrite(PIN_RELAY, LOW);
}

// Условие события: дым выше порога
bool conditionMet(float s1, float s2, float s3) {
  return s1 > THRESHOLD;
}

void setup() {
  Serial.begin(9600);
```

```
  pinMode(PIN_S2, INPUT);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  dht.begin();
  actuatorOff();
  digitalWrite(PIN_LED, LOW);
}

void loop() {
  n++;

  float s1 = readS1();
  float s2 = readS2();
  float s3 = readS3();

  Serial.print("DATA n=");   Serial.print(n);
  Serial.print(" s1=");      Serial.print(s1, 2);
  Serial.print(" s2=");      Serial.print((int)s2);
  Serial.print(" s3=");      Serial.println(s3, 1);

  bool cond = conditionMet(s1, s2, s3);

  if (cond && !alarm) {            // порог начал выполняться
    alarm = true;
    actuatorOn();
    digitalWrite(PIN_LED, HIGH);
    Serial.print("EVENT n=");  Serial.print(n);
    Serial.println(" type=THRESHOLD");
  } else if (!cond && alarm) {     // порог перестал выполняться
    alarm = false;
    actuatorOff();
    digitalWrite(PIN_LED, LOW);
    Serial.print("EVENT n=");  Serial.print(n);
    Serial.println(" type=NORMAL");
  }

  delay(T_MS);
}
```

Приложение Е Листинг функции разбора журнала Node-RED

Функция входит в узел function потока Node-RED. Полный экспорт потока nodered/flow.json размещён в репозитории проекта:

https://github.com/volan11/kyrsovaya21

```
// Вариант 21. Санузел. Разбор журнала Serial Monitor и подготовка SQL
const T = 1;                                    // период опроса, с
const RANGES = { s1: [0, 100], s2: [0, 1], s3: [0, 100] };  // допустимые диапазоны

const text = String(msg.payload).replace(/^/, '');
const sql = [];

text.split(/\r?\n/).forEach(raw => {
    const line = raw.trim();
    if (!line) return;
    const parts = line.split(' ');
    const kind = parts[0];                      // DATA или EVENT
    const f = {};
    parts.slice(1).forEach(p => {
        const kv = p.split('=');
        f[kv[0]] = kv[1];
    });
    const n = parseInt(f.n);
    if (isNaN(n)) return;
    const t = n * T;

    if (kind === 'DATA') {
        ['s1', 's2', 's3'].forEach(s => {
            const v = parseFloat(f[s]);
            // 1) запись в readings (если значения нет - пропускаем)
            if (!isNaN(v)) {
                sql.push(`INSERT INTO readings(n,t_s,sensor,value) VALUES(${n},$
{t},'${s}',${v});`);
            }
            // 2) значение отсутствует или вне диапазона - неисправность
            if (isNaN(v) || v < RANGES[s][0] || v > RANGES[s][1]) {
                sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},$
{t},'FAULT','датчик ${s} вне диапазона');`);
            }
        });
    } else if (kind === 'EVENT') {
        sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'$
{f.type}',NULL);`);
    }
});

const head = 'BEGIN;\nDELETE FROM readings;\nDELETE FROM events;\n' +
    "DELETE FROM sqlite_sequence WHERE name IN ('readings','events');\n";
return [null, { payload: head + sql.join('\n') + '\nCOMMIT;\n' }];
```

Приложение Ж Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings (
  id     INTEGER PRIMARY KEY AUTOINCREMENT,
  n      INTEGER NOT NULL,
  t_s    INTEGER NOT NULL,
  sensor TEXT    NOT NULL,
  value  REAL    NOT NULL
);

CREATE TABLE IF NOT EXISTS events (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  n       INTEGER NOT NULL,
  t_s     INTEGER NOT NULL,
  type    TEXT    NOT NULL,
  message TEXT
);
```

db/queries.sql

```
-- Контрольные запросы к базе db/variant.db (вариант 21, Санузел)

-- 1. Десять последних показаний (таблица readings).
SELECT id, n, t_s, sensor, value FROM readings ORDER BY id DESC LIMIT 10;

-- 2. Минимум, среднее и максимум по каждому датчику (таблица readings).
SELECT sensor, MIN(value) AS min_v, ROUND(AVG(value), 2) AS avg_v, MAX(value) AS
max_v
FROM readings GROUP BY sensor ORDER BY sensor;

-- 3. Все события по порядку времени (таблица events).
SELECT id, n, t_s, type, message FROM events ORDER BY t_s, id;

-- 4. Количество записей в каждой таблице.
SELECT 'readings' AS tbl, COUNT(*) AS cnt FROM readings
UNION ALL
SELECT 'events', COUNT(*) FROM events;
```

Приложение И Журналы Serial, загрузочный скрипт и результаты запросов

Журнал data/log.txt (штатный сценарий)

```
DATA n=1 s1=80.50 s2=0 s3=55.0
EVENT n=1 type=THRESHOLD
DATA n=2 s1=80.50 s2=0 s3=55.0
DATA n=3 s1=80.50 s2=0 s3=55.0
DATA n=4 s1=80.50 s2=0 s3=55.0
DATA n=5 s1=80.50 s2=0 s3=55.0
DATA n=6 s1=80.50 s2=0 s3=55.0
DATA n=7 s1=80.50 s2=0 s3=55.0
DATA n=8 s1=80.50 s2=0 s3=55.0
DATA n=9 s1=80.50 s2=0 s3=55.0
DATA n=10 s1=80.50 s2=0 s3=55.0
DATA n=11 s1=0.00 s2=0 s3=55.0
EVENT n=11 type=NORMAL
DATA n=12 s1=0.00 s2=0 s3=55.0
DATA n=13 s1=0.00 s2=0 s3=55.0
DATA n=14 s1=0.00 s2=0 s3=55.0
DATA n=15 s1=0.00 s2=0 s3=55.0
DATA n=16 s1=0.00 s2=0 s3=55.0
DATA n=17 s1=0.00 s2=0 s3=55.0
DATA n=18 s1=0.00 s2=0 s3=55.0
DATA n=19 s1=0.00 s2=0 s3=55.0
DATA n=20 s1=0.00 s2=0 s3=55.0
DATA n=21 s1=0.00 s2=0 s3=55.0
DATA n=22 s1=0.00 s2=0 s3=55.0
DATA n=23 s1=0.00 s2=0 s3=55.0
DATA n=24 s1=0.00 s2=0 s3=55.0
DATA n=25 s1=0.00 s2=0 s3=55.0
DATA n=26 s1=0.00 s2=0 s3=55.0
DATA n=27 s1=0.00 s2=0 s3=55.0
DATA n=28 s1=0.00 s2=0 s3=55.0
DATA n=29 s1=0.00 s2=0 s3=55.0
DATA n=30 s1=0.00 s2=0 s3=55.0
DATA n=31 s1=0.00 s2=1 s3=55.0
DATA n=32 s1=0.00 s2=1 s3=55.0
DATA n=33 s1=0.00 s2=1 s3=55.0
DATA n=34 s1=0.00 s2=1 s3=55.0
DATA n=35 s1=0.00 s2=1 s3=55.0
DATA n=36 s1=0.00 s2=0 s3=55.0
DATA n=37 s1=0.00 s2=0 s3=55.0
DATA n=38 s1=0.00 s2=0 s3=55.0
DATA n=39 s1=0.00 s2=0 s3=55.0
DATA n=40 s1=0.00 s2=0 s3=55.0
DATA n=41 s1=0.00 s2=0 s3=55.0
DATA n=42 s1=0.00 s2=0 s3=55.0
DATA n=43 s1=0.00 s2=0 s3=55.0
DATA n=44 s1=0.00 s2=0 s3=55.0
DATA n=45 s1=0.00 s2=0 s3=55.0
DATA n=46 s1=0.00 s2=0 s3=55.0
DATA n=47 s1=0.00 s2=0 s3=55.0
DATA n=48 s1=0.00 s2=0 s3=55.0
DATA n=49 s1=0.00 s2=0 s3=55.0
DATA n=50 s1=0.00 s2=0 s3=55.0
DATA n=51 s1=0.00 s2=0 s3=55.0
DATA n=52 s1=0.00 s2=0 s3=55.0
DATA n=53 s1=0.00 s2=0 s3=55.0
DATA n=54 s1=0.00 s2=0 s3=55.0
DATA n=55 s1=0.00 s2=0 s3=55.0
DATA n=56 s1=0.00 s2=0 s3=55.0
DATA n=57 s1=38.82 s2=0 s3=55.0
```

```
EVENT n=57 type=THRESHOLD
DATA n=58 s1=38.82 s2=0 s3=55.0
DATA n=59 s1=38.82 s2=0 s3=55.0
DATA n=60 s1=38.82 s2=0 s3=55.0
DATA n=61 s1=38.82 s2=0 s3=55.0
DATA n=62 s1=38.82 s2=0 s3=55.0
DATA n=63 s1=38.82 s2=0 s3=55.0
DATA n=64 s1=38.82 s2=0 s3=55.0
DATA n=65 s1=38.82 s2=0 s3=55.0
DATA n=66 s1=38.82 s2=0 s3=55.0
DATA n=67 s1=38.82 s2=0 s3=93.5
DATA n=68 s1=38.82 s2=0 s3=93.5
DATA n=69 s1=38.82 s2=0 s3=93.5
DATA n=70 s1=38.82 s2=0 s3=93.5
DATA n=71 s1=38.82 s2=0 s3=93.5
DATA n=72 s1=38.82 s2=0 s3=93.5
DATA n=73 s1=38.82 s2=0 s3=93.5
DATA n=74 s1=38.82 s2=0 s3=93.5
DATA n=75 s1=38.82 s2=0 s3=93.5
DATA n=76 s1=38.82 s2=0 s3=93.5
DATA n=77 s1=38.82 s2=0 s3=93.5
DATA n=78 s1=0.00 s2=0 s3=93.5
EVENT n=78 type=NORMAL
DATA n=79 s1=0.00 s2=0 s3=93.5
DATA n=80 s1=0.00 s2=0 s3=93.5
DATA n=81 s1=0.00 s2=0 s3=93.5
DATA n=82 s1=0.00 s2=0 s3=93.5
DATA n=83 s1=0.00 s2=0 s3=93.5
DATA n=84 s1=0.00 s2=0 s3=93.5
DATA n=85 s1=0.00 s2=0 s3=93.5
DATA n=86 s1=0.00 s2=0 s3=93.5
DATA n=87 s1=0.00 s2=0 s3=93.5
DATA n=88 s1=0.00 s2=0 s3=93.5
DATA n=89 s1=0.00 s2=0 s3=93.5
DATA n=90 s1=0.00 s2=0 s3=93.5
DATA n=91 s1=0.00 s2=0 s3=93.5
```

Журнал data/log_fault.txt (проверка неисправности)

```
DATA n=1 s1=999 s2=1 s3=55.0
```

Файл db/load.sql, сформированный Node-RED для log_fault.txt

```
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN ('readings','events');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,1,'s1',999);
INSERT INTO events(n,t_s,type,message) VALUES(1,1,'FAULT','датчик s1 вне
диапазона');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,1,'s2',1);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,1,'s3',55);
COMMIT;
```

Результаты контрольных запросов (sqlite3 db/variant.db ".read db/queries.sql")

```
-- 1. Десять последних показаний
id   n   t_s  sensor  value
---  --  ---  ------  -----
273  91  91   s3      93.5
272  91  91   s2      0.0
271  91  91   s1      0.0
270  90  90   s3      93.5
```

```
269  90  90   s2      0.0
268  90  90   s1      0.0
267  89  89   s3      93.5
266  89  89   s2      0.0
265  89  89   s1      0.0
264  88  88   s3      93.5
```

-- 2. Минимум, среднее и максимум по каждому датчику

```
sensor  min_v  avg_v  max_v
------  -----  -----  -----
s1      0.0    17.8   80.5
s2      0.0    0.05   1.0
s3      55.0   65.58  93.5
```

-- 3. Все события по порядку времени

```
id  n   t_s  type       message
--  --  ---  ---------  -------
1   1   1    THRESHOLD  null
2   11  11   NORMAL     null
3   57  57   THRESHOLD  null
4   78  78   NORMAL     null
```

-- 4. Количество записей в каждой таблице

```
tbl       cnt
--------  ---
readings  273
events    4
```

Приложение К История Git

```
$ git log --oneline

2fc2ee5 tmp

1dbb28f Журнал тестов и проверка неисправности FAULT

a2e7ce8 Схема базы данных SQLite и поток Node-RED для загрузки журнала

34219fa Модель Wokwi, прошивка variant_21.ino и журнал работы системы

77674d1 Структура проекта, README, схема электрическая принципиальная и перечень элементов
```